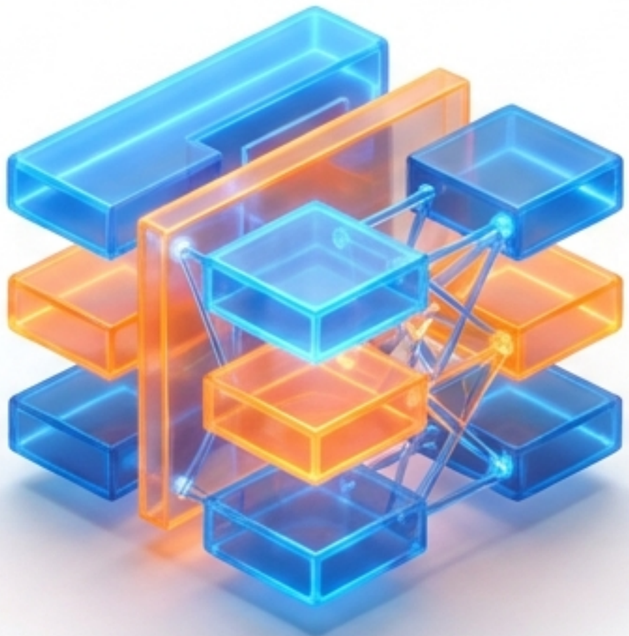
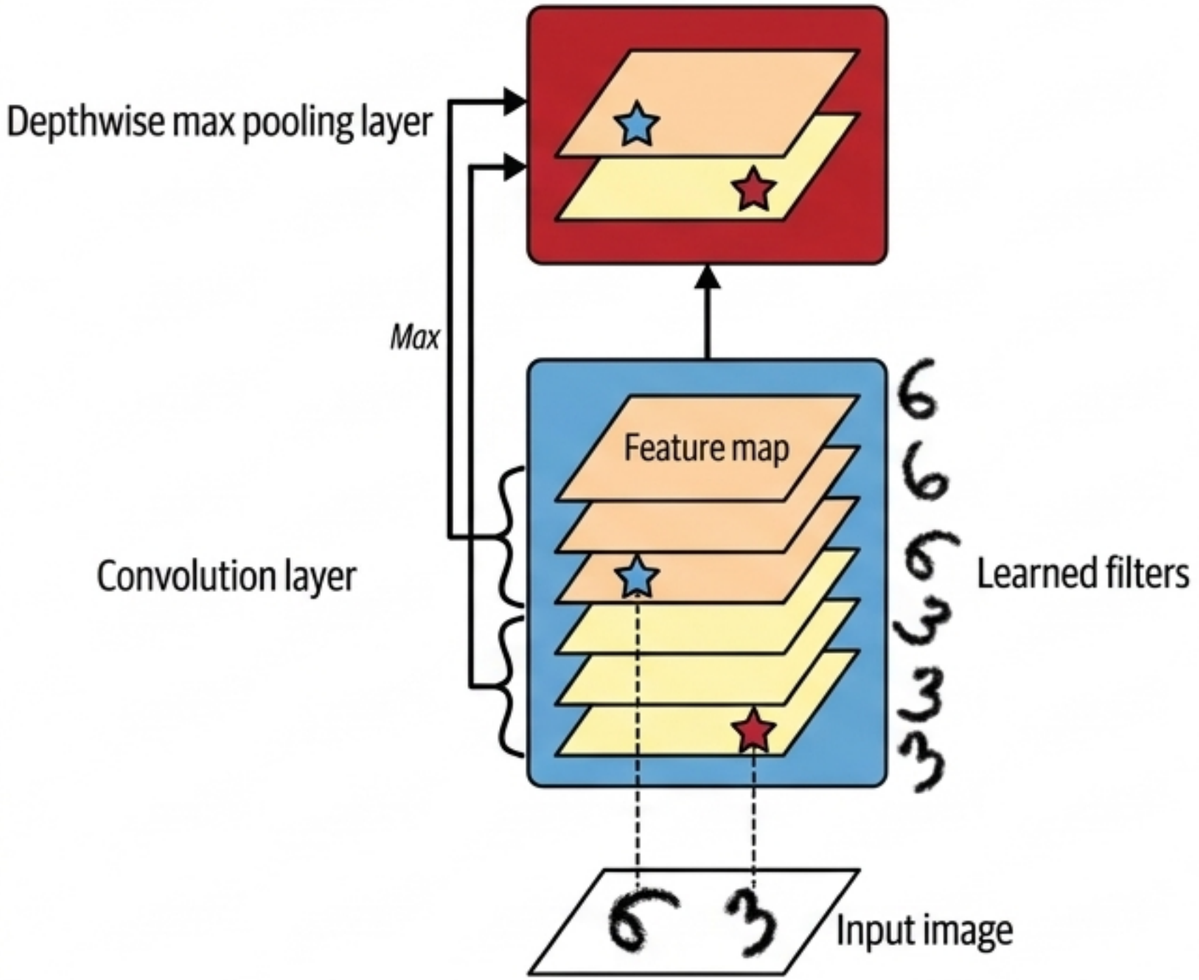


# Kiến trúc Mạng Nơ-ron Tích chập (CNN)

## Sự phát triển và Các mô hình Kinh điển

Gemini Notebook




*Hình 14-11: Tính bất biến nhờ Max Pooling*

# Tính Bất Biến đối với Dịch Chuyển (Translation Invariance)

Hình 14-11 minh họa: Chữ số C dịch sang phải vài pixel.

Ở các lớp dưới, Max Pooling bảo toàn đường nét chính dù có xê dịch.

Kết quả là vector đặc trưng đầu ra của lớp cuối cùng gần như không đổi.

Điều này vô cùng có lợi cho phân loại, không cần phải dạy mạng học từng vị trí của con mèo trong ảnh.

Gemini Notebook

# Tổng hợp Kiến Trúc CNN Tiêu Chuẩn

Hình 14-12 tổng hợp một quy trình chuẩn:




| 1 | 2 | 3 | 4 | 5 |
|---|---|---|---|---|
| Cụm (Conv + ReLU + Conv + ReLU + MaxPool). Lặp lại vài lần. | Ảnh nhỏ dần nhưng Dày lên (nhiều feature maps). | Lớp Flatten: Duỗi thẳng thành vector 1D. | Các lớp Dense (Fully Connected) đóng vai trò bộ phân loại. | Output layer (Softmax). |

*Hình 14-12: Mạng Nơ-ron Tích chập điển hình*

Gemini Notebook

# Nền móng - LeNet-5 (1998)

- Tác giả: Yann LeCun.
- Ứng dụng: Nhận diện mã bưu điện viết tay trên phong bì.
- Kiến trúc kinh điển đầu tiên kết hợp Convolution và Average Pooling.
- Đặt nền móng cho cuộc cách mạng Deep Learning.

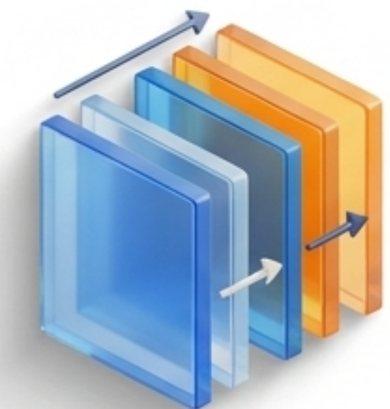



Gemini Notebook

# Đột phá - AlexNet (2012)



Chiến thắng vang dội tại cuộc thi **ImageNet ILSVRC 2012.**



Kiến trúc sâu hơn **LeNet-5.**



Lần đầu tiên sử dụng hàm kích hoạt **ReLU**, giúp mạng học cực nhanh.

Lần đầu tiên sử dụng kĩ thuật chống quá khớp đột phá là **Dropout.**



Chạy trên **2 GPU NVIDIA** vì GPU ngày đó không đủ **RAM** để chứa cả mô hình.

Gemini Notebook

# Tăng cường Dữ liệu (Data Augmentation)

AlexNet nổi tiếng với việc tạo ra dữ liệu giả khổng lồ từ dữ liệu thật. Hình 14-13 minh họa quá trình:



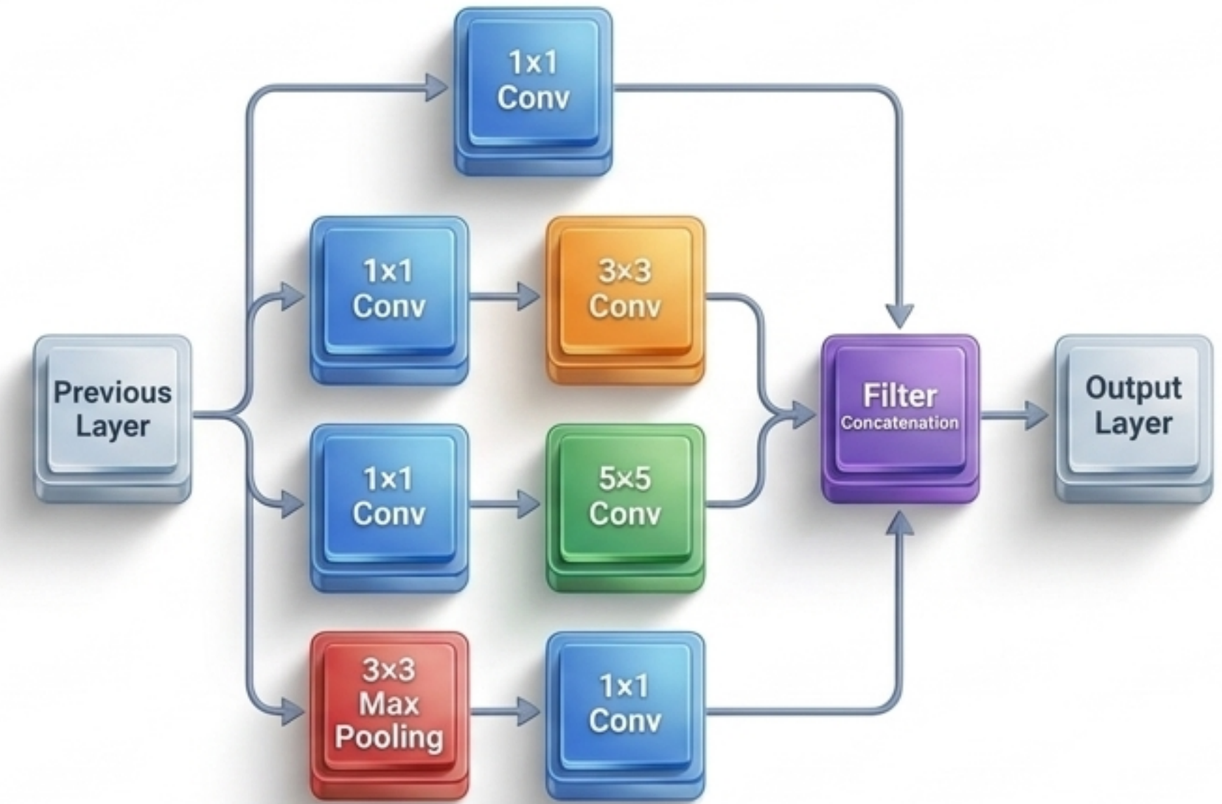

*Hình 14-13: Tăng cường dữ liệu (Data Augmentation)*

Từ 1 ảnh gốc con ếch, tạo ra hàng chục ảnh đã bị lật ngang, lật dọc, dịch chuyển, xoay, thay đổi màu sắc. Việc này giúp mô hình chống overfit và ép nó học các tính chất bất biến phức tạp hơn.

Gemini Notebook

# Mạng GoogLeNet (2014) và Mô đun Inception

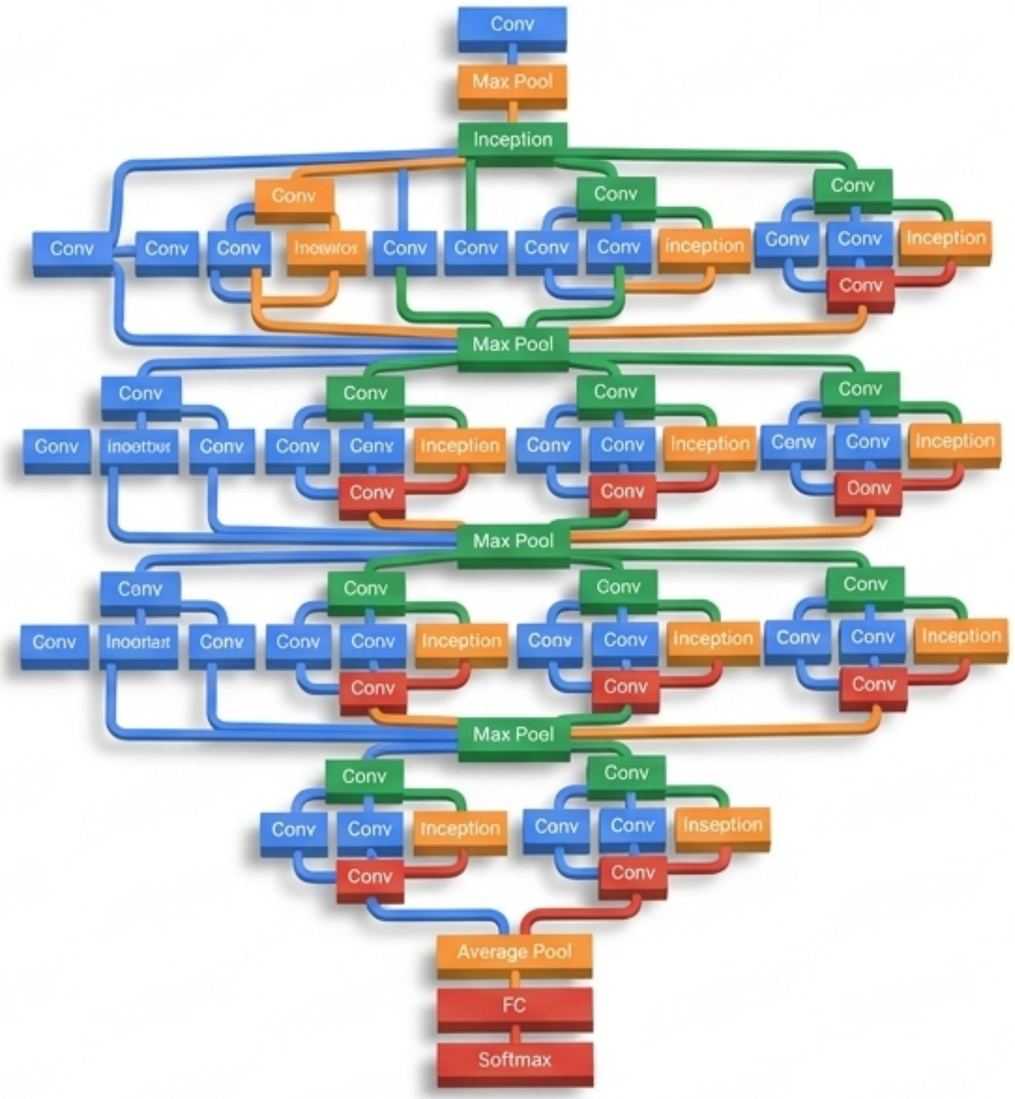
- Lỗi **10%** trên ImageNet xuống còn **<7%**.
- Bí quyết là các mô-đun **Khối Inception** (Hình 14-14).
- Thay vì hỏi: "Nên dùng Kernel 1x1, 3x3 hay 5x5?", GoogLeNet dùng TẤT CẢ cùng lúc trong 1 mô-đun.
- Đặc biệt sử dụng nhiều **Conv 1x1** để "bóp" số lượng kênh, giảm khối lượng tính toán.




*Hình 14-14: Kiến trúc một mô-đun Inception*

Gemini Notebook

# Mạng GoogLeNet Hoàn Chỉnh



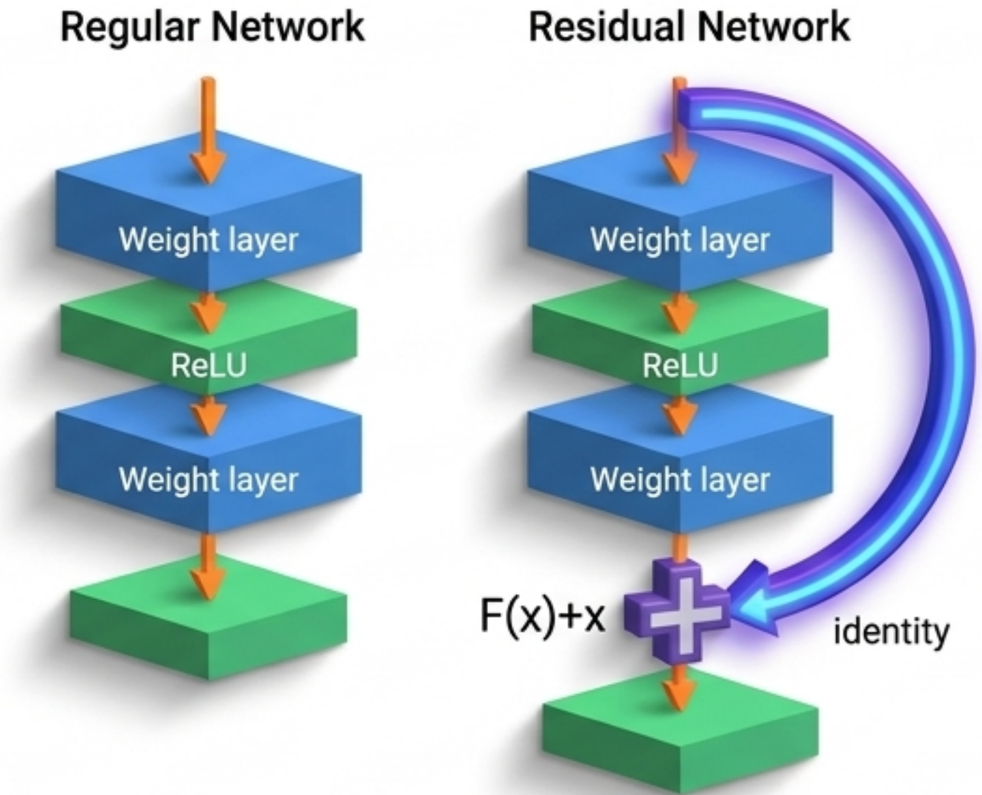

*Hình 14-15: Toàn cảnh Kiến trúc GoogLeNet sâu thẳm*

Gemini Notebook

# ResNet (2015) - Residual Networks



- Khi mạng quá sâu (hơn 30 **lớp**), lỗi huấn luyện lại tăng do Gradient suy thoái.
- ResNet giải quyết bằng Kết nối tắt (Skip Connections).
- Thay vì học trực tiếp một hàm $H(x)$, ResNet học phần dư $F(x) = H(x) - x$.
- Hình 14-16 minh họa khối dư (Residual block).




*Hình 14-16: Khối Residual Learning cơ bản*

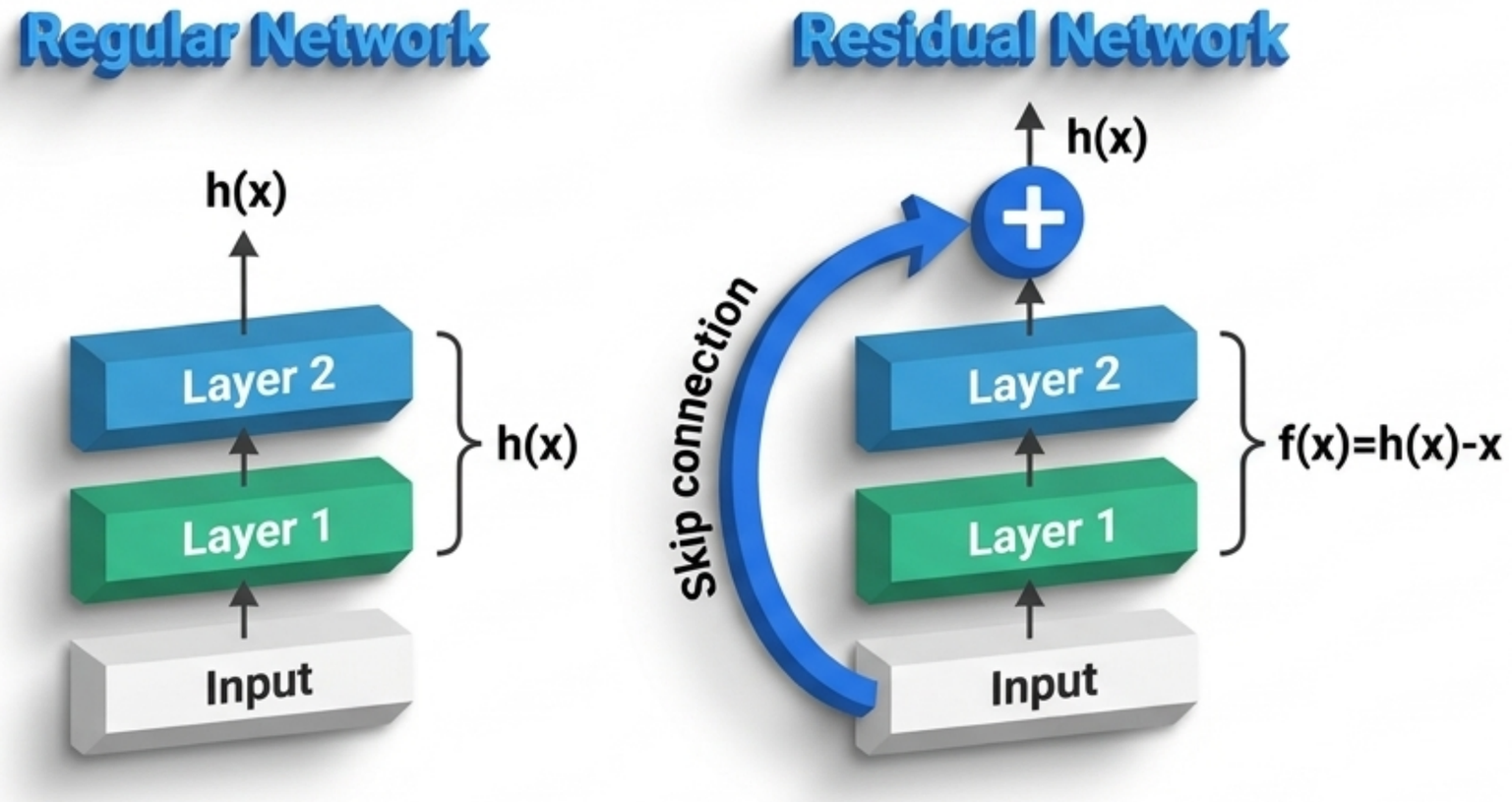
Gemini Notebook

# Cơ chế Hoạt động của Skip Connection

**Hình 14-17** so sánh **Mạng thường** (Regular Network) và **Mạng Dư thừa** (Residual Network).

**Lợi ích:** Dòng chảy gradient truyền trực tiếp qua nét liền x, không bị suy giảm.

Cho phép mạng sâu tới **152 lớp** (**ResNet-152**) mà huấn luyện vẫn hội tụ rất nhanh.




*Hình 14-17: Mạng lưới sâu với kết nối bỏ qua*

Gemini Notebook

# Toàn Cảnh Kiến Trúc ResNet

- **Hình 14-18:** Mạng ResNet xếp chồng hàng chục khối RU liên tục.
- Ở các điểm mốc (đường nét đứt), số kênh nhân đôi, bản đồ đặc trưng thu nhỏ lại một nửa (thông qua Stride=2).



*Hình*

Gemini Notebook

# Code: Xây dựng Khối Residual (ResNet Block)

```
class ResidualUnit(keras.layers.Layer):
    def __init__(self, filters, strides=1, activation="relu", **kwargs):
        super().__init__(**kwargs)
        self.activation = keras.activations.get(activation)
        self.main_layers = [
            keras.layers.Conv2D(filters, 3, strides=strides, padding="same", use_bias=False),
            keras.layers.BatchNormalization(),
            self.activation,
            keras.layers.Conv2D(filters, 3, strides=1, padding="same", use_bias=False),
            keras.layers.BatchNormalization()]
        self.skip_layers = []
        if strides > 1:
            self.skip_layers = [
                keras.layers.Conv2D(filters,1, strides=strides, padding="same", use_bias=False),
                keras.layers.BatchNormalization()]

    def call(self, inputs):
        Z = inputs
        for layer in self.main_layers: Z = layer(Z)
```

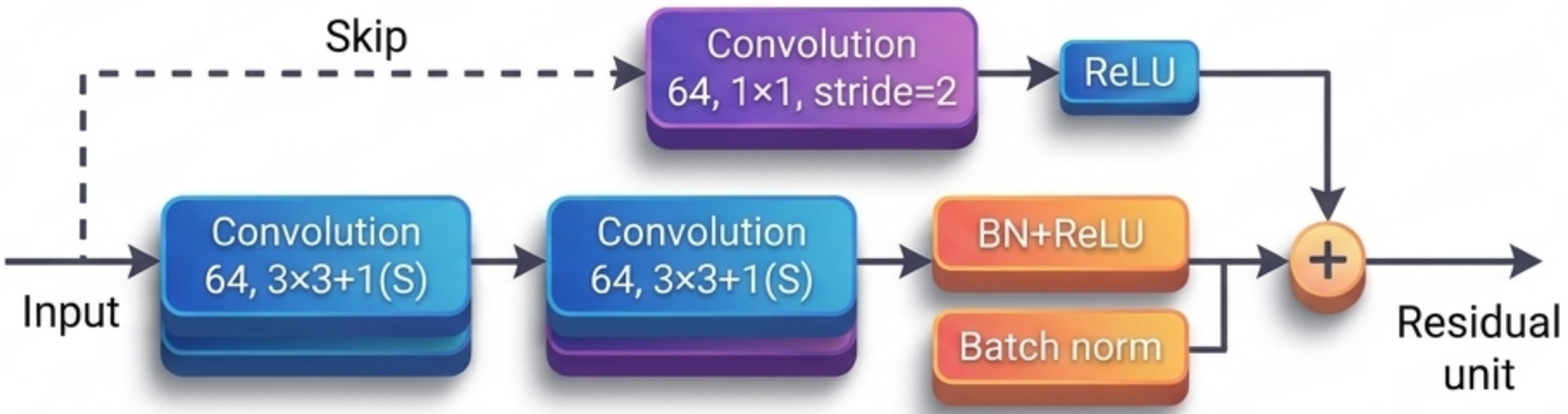
Gemini Notebook

# Xử lý Kích thước Kênh qua Skip Connection



- **Khi thay đổi số lượng bản đồ đặc trưng**, ta không thể cộng trực tiếp F(x) + x (Vì khác kích thước ma trận).
- **Giải pháp:** Thêm một lớp Conv 1x1 (stride=2) vào đường chéo (Skip connection) để đồng bộ số lượng bộ lọc.
- Hình 14-19 mô phỏng kĩ thuật này.



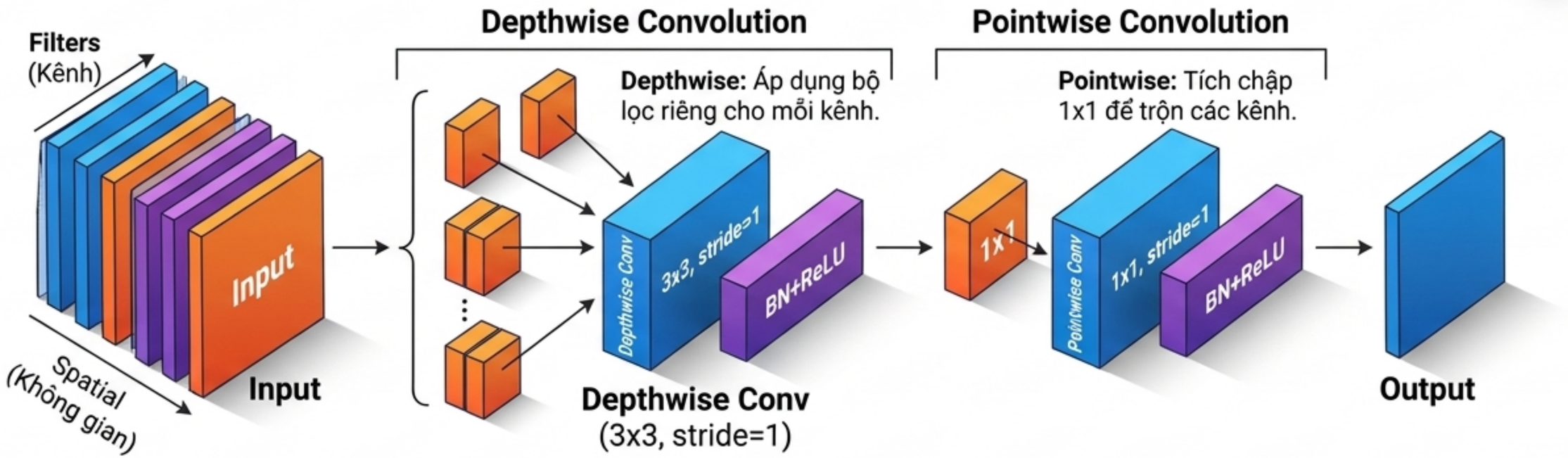



***Hình 14-19:*** *Khối thay đổi kích thước*

Gemini Notebook

# Kiến trúc Xception (2016)

- **Tác giả:** Francois Chollet (Cha đẻ của Keras). Tên gọi Xception = ”Extreme Inception”.
- Tách biệt hoàn toàn Không gian (Spatial) và Kênh (Cross-Channel).
- Sử dụng Depthwise Separable Convolution (Hình 14-20) và (Hình 14-21).
- **Ưu điểm:** Tốn ít bộ nhớ và tham số hơn Inception rất nhiều nhưng hiệu năng tốt hơn.




*Hình 14-20 & Hình 14-21*

Gemini Notebook

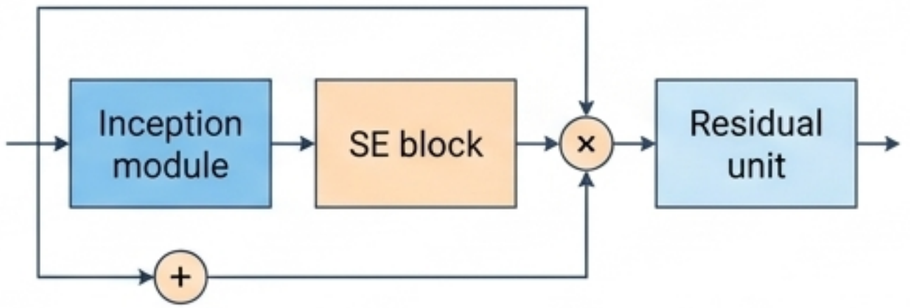
# Kiến trúc SENet (2017) - Squeeze-and-Excitation

- Đạt giải nhất ImageNet 2017.

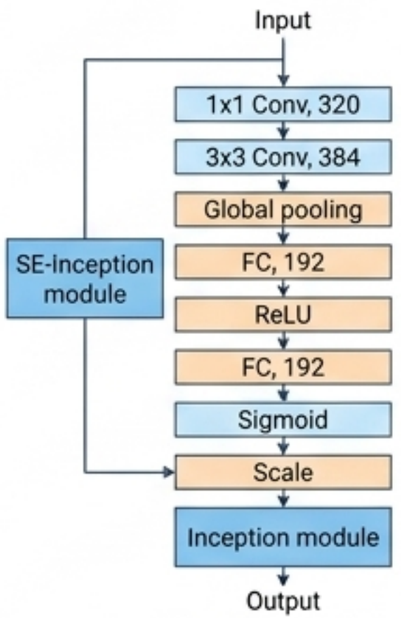
- Nâng cấp mọi mạng hiện có bằng việc gắn thêm Khối SE (Hình 14-22).

- Khối SE tự động học để "đánh trọng số" mức độ quan trọng cho các Bản đồ đặc trưng (kênh). (Vd: Nếu thấy cái mũi, lập tức tăng cường các bộ lọc của con mắt/miệng để tìm khuôn mặt).






**Hình 14-22:** Khối SE (Squeeze-and-Excitation)




**Hình 14-23:** Tích hợp SE block vào mô-đun Inception

Gemini Notebook